

第四章 贪心算法

Greedy

!!贪心不一定正确(0-1背包), 需要证明

4.1 活动安排问题

4.2 贪心算法的基本要素（分数背包）

最优装载

哈夫曼编码

多机调度

最小生成树

构造“贪心”反例

- 找零问题：一出纳员支付一定数量的现金。假设他手中有各种面值的纸币和硬币，要求他用最少的货币数支付规定的现金
- 例如，现有4种硬币：它们的面值分别为1分、2分、5分和1角，要支付2角5分
- 首先支付2个1角硬币，然后支付一个5分硬币，这就是贪心策略
- 反例：三种硬币：1分、4分、6分，支付8分
贪心3个硬币，最优2个硬币

货郎担问题

- 设售货员要到五个城市去售货，最后再回到出发的城市。已知从一个城市到其他城市的费用，求总费用最少的路线

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

问题

- 采用贪婪算法从城市1出发，如何选择路线和所需要的最少费用是多少？
- 采用贪婪算法从城市2出发，如何选择路线和所需要的最少费用是多少？
- 采用贪婪算法从城市3出发，如何选择路线和所需要的最少费用是多少？

货郎担问题(续)

- 解：下面给出称为登山法的算法。
所谓登山法，就是从初始结点出发，满足某些约束条件，一步一步地“登到”终点。
设 n 阶无向带权完全图 $G=<V,E,W>$
 - (1) 以 v_i 为起点，在其余的 $n-1$ 个结点中，找出最邻近于 v_i 的结点 v_j （ v_j 可能不唯一，但只选一个），形成初级通路 $v_i v_j$
 - (2) 在其余的 $n-2$ 个城市中找出最邻近于 v_j 的结点 v_k ，形成初级通路 $v_i v_j v_k$

货郎担问题(续)

- (3) 如此进行下去，直到将所有结点都扩充到通路中来，最后得到长度为 n 的初级通路就是图中一条哈密顿回路，将它的权做为图中最短哈密顿回路长度的近似值。这个算法就是登山法。

货郎担问题(续)

采用贪婪算法从城市**1**出发，选择的路线是：

1->4 ->3 -> 5->2 ->1。总费用为**14**

而最优解的路线是：

1->2 ->5 -> 4->3 ->1。总费用为**13**

这说明贪婪算法解货郎担问题时，不具有最优子结构性，也不具有贪婪选择性质。因为贪婪法在局部状态下，选择边 e_{14} 作为部分解的因素时，**并没有考虑到将来的选择是否仍可以达到最优**。因此，无法保证边 e_{14} 是全局最优解的元素。

贪心算法与动态规划算法的差异

- 贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。
- 但是，对于具有最优子结构的问题应该选用贪心算法还是动态规划算法求解？是否能用动态规划算法求解的问题也能用贪心算法求解？
- 2个经典的组合优化问题，并以此说明贪心算法与动态规划算法的主要差别。
 - 0-1背包问题
 - 背包问题

构造“贪心”反例：着色

- 1) 将图 G 中的结点按度数递减的次序进行排列(相同度数的结点的排列随意)。
- 2) 用第一种颜色，对第一点着色，并按排列次序对与前面结点不相邻的每一点着同样的颜色。
- 3) 用第二种颜色对尚未着色的点重复第 2 步, 直到所有的点都着上颜色为止。

构造“贪心”反例：着色

解：

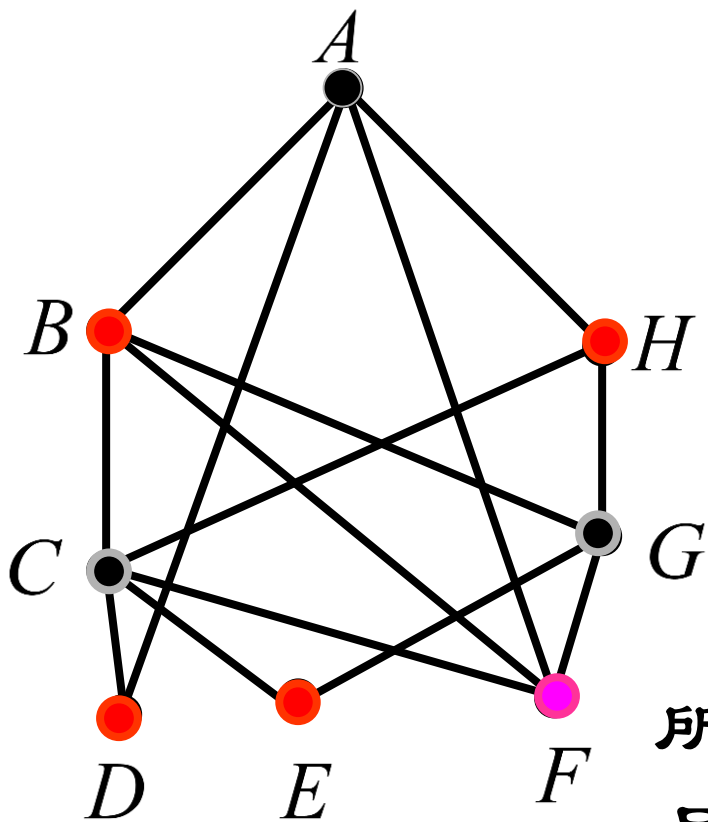
- 按度数递减次序排列各点

~~C~~ ~~A~~ ~~B~~ ~~F~~ ~~G~~ ~~H~~ ~~D~~ ~~E~~

- 第一种颜色： C, A, G
- 第二种颜色： B, H, D, E
- 第三种颜色： F

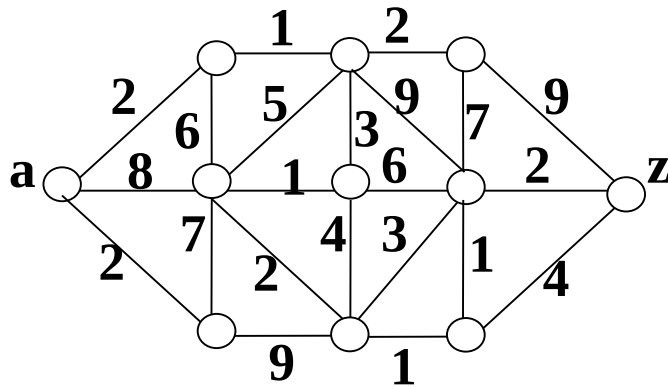
所以图是三色的。

另外图不能是两色的,因为图中有A,B,F两两相邻,所以 $\chi(G)=3$



最短路问题

小张被借调到一个新单位，图中**a**点是小张的住宅，**z**为新单位的位置，边上的数字表示距离，则小张到新单位的最短距离为___.



13

问题引入

- 有一类这样的问题，它有 n 个输入，而它的解由这 n 个输入的某个子集组成（可行解），只是这个子集必须满足某些事先给定的条件（约束条件）。
- 为了衡量可行解的优劣事先也给出一定的标准（目标函数）。那些使目标函数取极值（极大或极小）的可行解，称为最优解。

问题引入

对于这一类需求最优解的问题，又可根据约束条件和目标函数的数学模型的特性或求解问题方法的不同进而细分为线性规划、整数规划、非线性规划和动态规划等问题。

- 贪心法是一种改进了的分级处理方法。它首先根据题意，选取一种量度标准，然后将这 n 个输入排成这种量度标准所要求的顺序，按这种顺序一次输入一个量。

问题引入

- 如果这个输入和当前已构成在这种量度意义下的部分最优解加在一起不能产生一个可行解，则不把此输入加到这部分解中。这种能够得到某种量度意义下的分级处理方法称为贪心法。

贪心算法

- 顾名思义，**贪心算法**总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优选择**。

贪心算法

- 当然，希望贪心算法得到的最终结果也是整体最优的。虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。
- 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

贪心算法的基本要素

- 对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。
- 但是，从许多可以用贪心算法求解的问题中看到这类问题一般具有2个重要的性质：**贪心选择性质**和**最优子结构性**质。

贪心选择性质

- 所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。
- 这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

贪心选择性质

- 动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，**每作一次贪心选择就将所求问题简化为规模更小的子问题。**

贪心选择性质（续）

- 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

最优子结构性性质

- 问题的最优子结构性性质是该问题可用动态规划算法或贪心算法求解的关键特征。

贪心算法的基本要素

对于一个具体问题, 怎么知道能否用贪心算法

- 贪心选择性质和最优子结构性质(比较矩阵连乘)
- 贪心选择性质

对比: 矩阵连乘, 0-1背包, 分数背包

贪心算法第一基本要素, 与DP主要区别

自顶向下计算

- **OSP**: 最优策略的子策略也是最优 //动规, 贪心
- 正确性证明一般过程:

贪心选择+OSP+数学归纳法

条件: 子问题与原问题类似, 相对独立

子问题的最优解和贪心选择联合得整体最优解

第四章 贪心算法

Greedy

!!贪心不一定正确(0-1背包), 需要证明

4.1 活动安排问题

4.2 贪心算法的基本要素（分数背包）

最优装载

哈夫曼编码

多机调度

最小生成树

应用：活动安排问题

活动安排: 问题描述

● 贪心策略 将各活动按**结束时间**排序 $f_1 \leq f_2 \leq \dots \leq f_n$, 先选出活动 1, 然后按活动编号从小到大的次序依次选择与当前活动相容的活动。

注: 这种策略使剩余的可安排时间极大化, 以便于安排尽可能多的相容活动。

活动安排: 贪心算法

```
void ActivitySelection(int n,s[],f[], bool a[])
```

{//f[]已排序, a[]记录选择的活动, 即a[i]=true表示活动i已选择

```
    a[1]=true;
```

```
    int j=1;
```

```
    for(int i=2; i<=n; i++) {
```

```
        if(s[i]>=f[j]) {
```

```
            a[i]=true;
```

```
            j=i;}
```

```
        else a[i]=false;
```

```
    }
```

```
}
```

活动安排算法正确性证明

n 个活动申请一个活动室, 活动起始终止区间 (s_i, f_i)

- 输入: $n, (s_i, f_i), i = 1:n$, 输出: 最大相容活动子集

- 第一步 证明贪心选择性质:

存在最优解包含相容最早结束的活动1

- 第二步 证明最优子结构性质:

若A是包含活动1的最大相容活动集(最优策略),

则A-{1}(子决策)是 $\{i \mid s_i \geq f_1\}$ (子问题)上的最大相容活动集

- 由此用数学归纳法, 就可以证明算法正确性

活动安排算法正确性证明

11个活动已按结束时间排序，用贪心算法求解：

i	1	2	3	4	5	6	7	8	9	10	11
start_time _i	1	3	0	5	3	5	6	8	8	2	12
finish_time _i	4	5	6	7	8	9	10	11	12	13	14

[illegible]

相容活动： $\{a_3, a_9, a_{11}\}$,
 $\{a_1, a_4, a_8, a_{11}\}$, $\{a_2, a_4, a_9, a_{11}\}$

[illegible]

活动安排算法正确性证明

最优解: $\{a_2, a_4, a_9, a_{11}\}$
 $\{a_1, a_4, a_8, a_{11}\}$

time	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
0											
1											
2											
3											
4											
5											
6											
7											
8											
9											
10											
11											
12											
13											
14											

第一步 证明贪心选择性质:

存在最优解包含相容最早结束的活动1

第二步 证明最优子结构性质:

若A是包含活动1的最大相容活动集(最优策略), 则A-{1}(子决策)是 $\{i \mid s_i \geq f_1\}$ (子问题)上的最大相容活动集

- 一、最小的1可以在里面, $\{1, 4, 9, 11\}$
- 则A-{1}={4, 9, 11}是子问题{4,6,7,8,9,11}的最优解
- 二、最小的4可以在里面, $\{4, 9, 11\}$
- 则A-{4}={9, 11}是子问题{8,9,11}的最优解
- 三、最小的8可以在里面, $\{8, 11\}$
- 则A-{8}={11}是子问题{11}的最优解

活动安排算法DP

设 $f_1 \leq f_2 \leq \dots \leq f_n$, 添加 $f_0 = 0, s_{n+1} = \infty$

$dp[i, j]$ = 在活动*i*结束后和活动*j*开始前的最大相容活动的活动个数

■ 输出 $dp[0, n+1]$

$$dp[i, j] = \begin{cases} 0 & \text{若为 } \phi \\ \max_{i < k < j} \{dp[i, k] + dp[k, j] + 1\} & \text{否则} \end{cases}$$

- 输出 $dp[0, n+1]$ 相容活动: $\{a_3, a_9, a_{11}\}$,
 $\{a_1, a_4, a_8, a_{11}\}$, $\{a_2, a_4, a_9, a_{11}\}$

$$dp[i, j] = \begin{cases} \max_{i < k < j} \{dp[i, k] + dp[k, j] + 1\} & \text{否则} \end{cases}$$

time	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
0											
1											
2											
3											
4											
5											
6											
7											
8											
9											
10											
11											
12											
13											
14											

- $dp[0, 12]$
 $= \max\{ dp[0, 1] + dp[1, 12] + 1,$
 $\dots$
 $dp[0, 3] + dp[3, 12] + 1,$
 $\dots$
 $dp[0, 8] + dp[8, 12] + 1,$
 $\dots$
 $\}$

应用：多机调度问题

多机调度问题

多机调度问题要求给出一种作业调度方案，使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。

约定，每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理。作业不能拆分成更小的子作业。

这个问题是**NP完全问题**，到目前为止还没有有效的解法。对于这一类问题，用**贪心选择策略**有时可以设计出较好的近似算法

问题

- 如何进行贪心选择？
- 采用最长处理时间作业优先的贪心选择策略可以设计出解多机调度问题的较好的近似算法

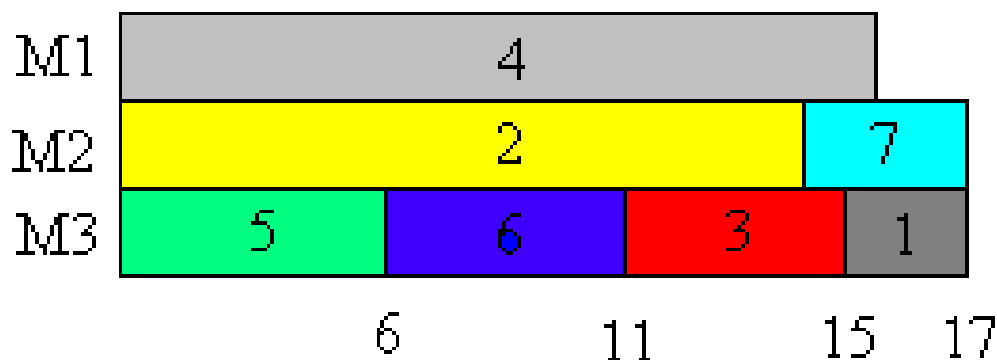
多机调度问题

例如， 设7个独立作业 $\{1, 2, 3, 4, 5, 6, 7\}$ 由3台机器M1, M2和M3加工处理。各作业所需的处理时间分别为 $\{2, 14, 4, 16, 6, 5, 3\}$ 。

多机调度问题

例如， 设7个独立作业{1, 2, 3, 4, 5, 6, 7}由3台机器M1, M2和M3加工处理。各作业所需的处理时间分别为{2, 14, 4, 16, 6, 5, 3}。

按算法greedy产生的作业调度如下图所示，所需的加工时间为17。



多机调度问题

- 当 $n \leq m$ 时，只要将机器 i 的 $[0, t_i]$ 时间区间分配给作业 i 即可，算法只需要 $O(1)$ 时间。
(n 为作业数量， m 为机器数量)
- 当 $n > m$ 时，首先将 n 个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为 $O(n \log n)$ 。

第四章 贪心算法

Greedy

!!贪心不一定正确(0-1背包), 需要证明

4.1 活动安排问题

4.2 贪心算法的基本要素（分数背包）

最优装载

多机调度

哈夫曼编码

最小生成树

应用：分数背包

0-1背包问题与背包问题

- 0-1背包问题:

- 给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？
- 在选择装入背包的物品时，对每种物品 i 只有2种选择，即装入背包或不装入背包。不能将物品 i 装入背包多次，也不能只装入部分的物品 i 。

- 背包问题:

- 与0-1背包问题类似，所不同的是在选择物品 i 装入背包时，可以选择物品 i 的一部分，而不一定要全部装入背包， $1 \leq i \leq n$ 。

0-1背包问题与背包问题

- 这2类问题都具有最优子结构性质，极为相似，
- 但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。

分数背包

- 已知一个容量大小为M重量的背包和n种物品，物品i的重量为 w_i ，假定物品i的一部分 x_i 放入背包会得到 $v_i x_i$ 这么大的收益，这里， $0 \leq x_i \leq 1$ ， $v_i > 0$ ，采用怎样的装包方法才会使装入背包的物品总效益最大

- 考虑以下情况下的背包问题

$n=3, M=20, (v_1, v_2, v_3)=(25, 24, 15)$

$(w_1, w_2, w_3)=(18, 15, 10)$

问题

- 贪心选择的依据

背包问题的贪心算法

- 1)按效益值非增次序将物品依次放入背包

(x_1, x_2, x_3)	$\sum w_i x_i$	$\sum v_i x_i$
(1, 2/15, 0)	20	28.2

- 2)按物品重量的非降次序将物品依次放入背包

(0, 2/3, 1)	20	31
--------------------	-----------	-----------

- 3)按 v_i/w_i 的非增次序将物品依次放入背包

(0, 1, 1/2)	20	31.5
--------------------	-----------	-------------

算法：贪婪法求解背包问题

输入：背包载重量M,n个物体的价值v[]和重量w[]

输出：n个物体装入背包的份量x[]，背包中物体的总价值

**void GreedyKnapsack(int n, float M, float v[], float w[],
float x[])**

{

float q[n];

float m, p= 0;

for(int i = 0; i<n;i++)

/*计算物体的价值重量比*/

{ q[i] = v[i] / w[i];

x[i]=0;

}

Sort(n, v, w, q); /*按照物体的价值重量比对物体进行排序*/

```

m = M;                                /*背包的剩余载重量*/
for(i=1; i<=n; i++)
{
    if(w[i]<=m)                            /*优先装入价值重量比大的物体*/
    {
        x[i]=1; m-=w[i];
        p+= v[i]
    }
    else{                                /*最后一个物体的装入份量*/
        x[i] = m / w[i]
        p+= x[i] * v[i];
        break;
    }
}
}

```

背包问题的贪心算法(续)

算法的时间复杂性为 $O(n \log n)$

- 下面来证明使用此贪心算法所得到的贪心解是一个最优解。

背包问题的贪心算法(续)

- 证明的基本思想是：把这个贪心解与任一个最优解相比较，如这两个解不同，就去找开始不同的第一个 x_i ，然后设法用贪心解的这个 x_i 去代换最优解的那个 x_i ，并证明在分量作了代换之后其总效益与代换之前无任何损失。
- 反复进行这种代换，直到新产生的最优解与贪心解完全一样，从而证明了贪心解是最优解。

背包问题的贪心算法(续)

- 定理： 如果 $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$ ，
则此算法对于给定的背包问题实例生成一个最优解。

分数背包贪心法证明

Case1: 所有 $x_i = 1$ 。显然该解就是最优解。

Case2: 设 $X = (1, \dots, 1, x_j, 0, \dots, 0)$ $x_j \neq 1, 1 \leq j \leq n$ 。下证 X 就是最优解。设问题的最优解 $Y = (y_1, \dots, y_n)$, 则存在 k 使得 $y_k \neq x_k$ 的最小下标 (否则 $Y = X$, 得证)。

首先, 可以证明 $y_k < x_k$ 。(反证: 若 $y_k > x_k$, 则 $x_k \neq 1, \therefore k \geq j$

$$\Rightarrow \sum_{i=1}^n w_i y_i \geq \sum_{i=1}^k w_i y_i > \sum_{i=1}^k w_i x_i = \sum_{i=1}^j w_i x_i = c \therefore Y \text{不是可行解, 矛盾})$$

下面改造 Y 成为新解 $Z = (z_1, \dots, z_n)$, 并使 Z 仍为最优解。将 y_k 增加到 x_k , 从 $(y_{k+1}, \dots, y_n)$ 中减同样的重量使总量仍是 c 。即,

$$z_i = x_i \quad i = 1, 2, \dots, k; \quad \text{和} \quad \sum_{i=k+1}^n w_i y_i - w_k (z_k - y_k) = \sum_{i=k+1}^n w_i z_i$$

分数背包贪心法证明

$$\begin{aligned}\therefore \sum_{i=1}^n w_i z_i &= \sum_{i=1}^{k-1} w_i z_i + w_k z_k + \sum_{i=k+1}^n w_i z_i \\ &= \sum_{i=1}^{k-1} w_i y_i + w_k z_k + \left(\sum_{i=k+1}^n w_i y_i - w_k (z_k - y_k) \right) = \sum_{i=1}^n w_i y_i = c\end{aligned}$$

$$\begin{aligned}\therefore \sum_{i=1}^n v_i z_i &= \sum_{i=1}^n v_i y_i + (z_k - y_k) v_k - \sum_{i=k+1}^n (y_i - z_i) v_i \quad // \because Z \text{ 由 } Y \text{ 变得的} \\ &= \sum_{i=1}^n v_i y_i + (z_k - y_k) w_k v_k / w_k - \sum_{i=k+1}^n (y_i - z_i) w_i v_i / w_i \\ &\geq \sum_{i=1}^n v_i y_i + \left[(z_k - y_k) w_k - \sum_{i=k+1}^n (y_i - z_i) w_i \right] v_k / w_k \quad // \text{利用 } v_i / w_i \downarrow \\ &= \sum_{i=1}^n v_i y_i\end{aligned}$$

$\therefore Z$ 也是最优解, 且 $z_i = x_i \ i=1, \dots, k$; 重复上面过程 $\Rightarrow X$ 为最优解。

0-1背包问题

- 有5个物体，其重量分别为2,2,6,5,4，价值分别为6,3,7,4,6，背包的载重量为10，求装入背包的物体及其总价值。

0-1背包问题(续)

- 利用贪心算法得到的解为：装入背包的物体的最大价值为15，装入背包的物体为 $X=\{1,1,0,0,1\}$ 。
- 最优解：装入背包的物体的最大价值为16，装入背包的物体为 $X=\{1,1,1,0,0\}$ 。

0-1背包问题(续)

0-1背包问题却不能用贪心算法求解。

- 对于0-1背包问题，贪心选择不能得到最优解。因为它无法保证最终能将背包装满，部分闲置的背包空间使背包空间的价值降低了。

有限期的任务安排问题

- 用贪心法求解有限期的任务安排问题：假设只能在同一台机器上加工 n 个任务，每个任务 i 完成时间均是一个单位时间，又设每个任务 i 都有一个完成期限 $d_i > 0$ ，当且仅当任务 i 在它的期限截止以前被完成时，任务 i 才能获得 p_i 的效益，每个任务的期限从整个工序的开工开始计时，问应如何安排加工顺序，才能获得最大效益

- $n=6, (p_1, p_2, p_3, p_4, p_5, p_6) = (5, 25, 20, 30, 10, 15),$

$$(d_1, d_2, d_3, d_4, d_5, d_6) = (1, 5, 2, 3, 3, 2)$$

有限期的任务安排问题

- 任务 i 完成时间均是一个单位时间，当且仅当任务 i 在它的期限截止以前被完成时， i 才能获得 p_i 的效益

$n=6, (p_1, p_2, p_3, p_4, p_5, p_6) = (5, 25, 20, 30, 10, 15),$

$(d_1, d_2, d_3, d_4, d_5, d_6) = (1, 5, 2, 3, 3, 2)$

- 类似单价排序？
- 法一：按效益从大到小排序

任务	0	1	2	3	4	5	6
p_i	0	30	25	20	15	10	5
d_i	0	3	5	2	2	3	1

有限期的任务安排问题

- 任务 i 完成时间均是一个单位时间，当且仅当任务 i 在它的期限截止以前被完成时， i 才能获得 p_i 的效益

法二：按期限从大到小排序

任务	0	1	2	3	4	5	6
p_i	0	25	30	10	20	15	5
d_i	0	5	3	3	2	2	1

有限期的任务安排问题

- 任务 i 完成时间均是一个单位时间，当且仅当任务 i 在它的期限截止以前被完成时， i 才能获得 p_i 的效益

法三：时间槽 [C]

按效益从大到小排序

任务	1	2	3	4	5	6
p_i	30	25	20	15	10	5
d_i	3	5	2	2	3	1

1	2	3	4	5